

App UI: navigation, layout, and the screens you always need

Week 7, Lecture 1 · Design for Builders: Ship Beautiful Products as a Founder

Autumn 2026 · Founders, indie hackers, engineers

What we'll cover today

- **Navigation patterns:** sidebar, top bar, and bottom tabs, choosing the right one
- **The five screens every app needs:** sign-in, empty state, primary task, success, error
- **Loading and error states:** designing for uncertainty, not the happy path
- **Settings and account screens:** the most-skipped, most-trust-building screens you own

Navigation patterns

Three patterns, three use cases

Navigation choice follows two variables: number of top-level destinations and primary device.

- **Top bar** (tabs along the top): 2-5 destinations on desktop, dense content, rare switching
- **Sidebar** (left rail): 4-8 destinations, desktop-first, frequent destination switching
- **Bottom tabs** (mobile bar): 3-5 destinations, mobile-first, thumb reachable

One rule: pick the pattern before you design a single screen. Every screen inherits it.

Sidebar: when it earns its width

A sidebar trades screen width for persistent context.

- User needs to know where they are at all times (dashboards, admin tools)
- Destination list is stable and labeled (Settings, Analytics, Billing)
- Desktop is the primary device

Collapsible sidebars (icon-only state) let you reclaim width without hiding the pattern.

Mistake to avoid: a sidebar with more than eight items. That is a navigation problem, not a design problem.

Top bar: for focused, linear flows

Top bars (horizontal tabs) work when destinations are few and switching is infrequent.

- 2-5 items maximum before the bar breaks at narrow viewports
- Works well for secondary navigation inside a section (Profile, Settings, Billing)
- Can coexist with a sidebar as a secondary layer

Figma itself uses this pattern: left sidebar for pages, top bar for view modes.

Bottom tabs: the mobile native

Bottom tabs put primary destinations within thumb reach.

- 3-5 tabs; five is the practical maximum (iOS HIG and Material both set this limit)
- Icons alone work at five or fewer tabs; always add labels for three or fewer
- Active state must be unambiguous: filled icon, accent color, or both

The failure mode: putting secondary destinations in bottom tabs. Bottom tabs are for the primary five things a user does every day, not every feature you offer.

The five screens every app needs

Why five, not one

Founders prototype the happy path. Investors see three screens. Users see twenty.

Brad Frost (2016, Atomic Design) describes the move from template to page as the moment abstract structure becomes real content. The five screens below are the five templates that every app renders under real conditions:

1. Sign-in
2. Empty state
3. Primary task
4. Success
5. Error

Design all five before launch. Finding the problems in Figma costs thirty minutes. Finding them in production costs users.

Screen 1: sign-in

Sign-in is the first screen most users see. It sets trust.

- Email input, password input, submit button, minimum
- Secondary actions: forgot password, sign up, SSO (if applicable)
- Error state for wrong credentials must be designed alongside the default state

Trust signals to include: HTTPS lock signal is implicit; a minimal, clean layout is explicit. Clutter on a sign-in screen reads as untrustworthy. Refactoring UI (Wathan and Schoger, 2019): "The harder it is to look at, the harder it is to trust."

Screen 2: empty state

The empty state is the screen most users see first, and the screen most teams design last.

Three-part structure that works:

1. **Why it is empty** (a brief, honest label or illustration)
2. **What to do next** (a direct, specific headline)
3. **The action** (one primary button)

Avoid the blank-slate trap: an empty list with no explanation tells the user nothing. It signals the product is broken, not that they have not yet added data.

Empty state: bad vs. good

What most teams ship

No label, no call to action, no explanation.

Icon or small illustration

One-line explanation of what this area is for

(blank area)

What earns trust

"You haven't saved anything yet"

[Save your first item] button

Screen 3: primary task

The primary task screen is the reason the product exists. Design this one last, after you know what the other four screens constrain.

- Navigation pattern locks in here (sidebar, tabs, or bottom bar)
- Card components appear at scale: a list of cards is the most common primary-task layout
- Empty state and loading state are variants of this screen, not separate screens

This screen stress-tests your component library. If a component breaks here, it will break everywhere.

Screen 4: success

Success screens close the feedback loop. A user completed an action; the product must confirm it clearly.

- **Confirmation:** what was completed, in plain language
- **Summary:** what the user now has or what will happen next
- **Next action:** where to go from here (do not strand the user)

The most common failure: a success screen that only says "Done." Done is not enough. "Your invoice has been sent to alex@example.com" is enough.

Screen 5: error

Error screens are the most trust-critical screens you design.

Three mandatory elements:

1. **What went wrong** (not an error code, a sentence)
2. **Why it happened** (if you can explain it simply)
3. **What to do next** (a recoverable action or a way back)

"Something went wrong. Please try again." fails all three. "We could not connect to your payment processor. Check your card details or try a different card." passes all three.

Loading and error states

Design for uncertainty, not the happy path

Every network call has three outcomes: success, failure, and waiting. Most designers design one.

- **Loading state:** skeleton screen or spinner? Skeletons reduce perceived wait time by keeping layout stable
- **Error state:** inline (form validation) vs. page-level (network failure), use inline for recoverable errors, page-level for catastrophic ones
- **Partial failure:** what does a card look like when one field failed to load?

Figma best practice: build loading and error as variants of your primary component, not as separate frames.

Skeleton screens vs. spinners

Spinners are for short, bounded waits (under two seconds). Skeletons are for content that has a known structure.

- A skeleton screen holds the layout: cards stay where cards will be, text lines stay where text will be
- A spinner holds nothing: the layout collapses, then re-expands when content arrives
- For a list of cards, use a skeleton. For a modal submitting a form, use a spinner on the button.

The distinction is whether the user needs to orient spatially during the wait.

Settings and account screens

The screens founders skip

Settings screens are built last and reviewed never. They are also the screens a skeptical user reads before they trust your product with their data.

A trust-building settings screen:

- Groups related controls visibly (account, notifications, billing, security are common groups)
- Uses plain language for every label
- Makes destructive actions (delete account, revoke access) visually distinct and protected by a confirmation step

The test: hand your settings screen to someone who has never seen your product. Can they find how to change their password and how to cancel their account in under thirty seconds?

Account screen essentials

The account screen is a functional document, not a feature showcase.

Required elements:

- Display name and email, editable
- Password change flow (or SSO provider name if passwordless)
- Avatar or profile image
- Account deletion or deactivation (with confirmation modal)

Optional but trust-building:

- Last login timestamp and device
- Active sessions list
- Two-factor authentication toggle

Take-aways

1. Choose a navigation pattern before you draw a single screen. Every screen inherits it.
2. Design all five required screens before launch: sign-in, empty state, primary task, success, error. The happy path is one of five, not the only one.
3. Loading and error states are variants, not afterthoughts. Skeleton screens reduce perceived wait; inline errors reduce recovery friction.
4. Settings and account screens are trust documents. A skeptical user reads them before they commit to your product.



"The moment you move from a template to a page is the moment real content meets your system. That is where problems surface."

Brad Frost, Atomic Design, chapter 2 (2016)

What's next

- **Section (this week):** build a complete five-screen flow in Figma using your component library
- **Lecture 2:** component systems, variants, states, and reuse in depth
- **Reading:** Week 7 reading, app screens, states, and component systems

HW 3 due this week: landing page design.

Capstone out this week: launch package.